Giraffes (Solution)

Review Hirotaka Yoneda (square1001)

In this review, we explain solutions for the 4 subtasks of the task "Giraffes" in JOI Open Contest 2022.

In the beginning, we describe an outline of solutions. Please read this section if you want to read a short review. However, it seems difficult to fully understand the solutions by a short review. Except for those who have enough confidence in the ability of programming contests, for the first time, we recommend you read solutions for each subtask in Section 2 and later sections. We explain the solutions for Subtasks 1–3 in a relatively comprehensive way. On the other hand, the solution for full score is rather difficult. We explain it for advanced participants.

1 Brief outline of solutions

A brief outline of a solution for each subtask is as follows. Here we only explain a brief outline. See Section 2 and later sections for detailed explanations on solutions. In the following, the initial positions of the giraffes are written as $P_1, P_2, \ldots, P_N$, and the positions of the giraffes after rearrangement are written as $P'_1, P'_2, \ldots, P'_N$.

Subtask 1 We can solve this subtask by checking all possible positions of the giraffes after rearrangement. The time complexity is $O(N! \times N^3)$.

Subtask 2 For the conditions to hold, it is necessary that either one of $P'_1 = 1, P'_1 = N, P'_N = 1, P'_N = N$ is satisfied. By recursively enumerating the permutations satisfying this condition, we can solve this subtask in $O(4^N)$ time.

Subtask 3 Using the same idea as Subtask 2, we can solve this subtask by Dynamic Programming (DP). Let $dp[l][x][y]$ be "the minimum number of elements changed from P_i when $P'_x, P'_{x+1}, \ldots, P'_{x+l}$ is a permutation of $y, y+1, \ldots, y+l$ satisfying the condition." We calculate $dp[l][x][y]$ from smaller values of l. The time complexity of this solution is $O(N^3)$.

Full Score Solution Here we use the fact that, on average, the maximum number of unchanged elements(= $N-$(solution of this task)) is $O(\sqrt{N})$. In Subtask 3, the region described by $dp[l][X][Y]$ can be considered as a square region $X \leq x \leq X+l,\ Y \leq y \leq Y+l$. We slightly modify DP in Subtask 3. We put $dp[i][j][k]$ to be "the maximum length of a side of a square such that the point (i, P_i) is a vertex of direction j ($j = 0, 1, 2, 3$) of the square and the number of unchanged elements is k." We shall calculate the values of $dp[i][j][k]$. Since we have $O(\sqrt{N})$ possible values of k, we can solve this task in $O(N^{2.5})$ time on

average. We speed up the DP using plane sweep algorithm with segment trees. Then we can solve this task in $O(N^{1.5} \log N)$ time on average, and get the full score.

Note that for the solutions of Subtasks 1,2,3, we do not use the randomness of input data. However, for the full score solution, we use the randomness of input data. In the worst case, the time complexity of the full score solution becomes $O(N^2 \log N)$.

2 Subtask 1 ($N \leq 7$)

There are $N! = 1 \times 2 \times \cdots \times N$ permutations of the giraffes. We can obtain the correct answer if we search all the possible permutations and, among arrangements of the giraffes satisfying the condition of the visual quality, calculate the minimum number of giraffes which are moved.

How can we write a program to search all the $N!$ permutations?[*1] There are several ways to search the permutations. The following are the major techniques.

- Call functions recursively to search all the possibilities.
- Use `std::next_permutation` to search all the possibilities.

Here we explain how to search all the possibilities using C++ standard library `std::next_permutation`. It is a rather easy technique. This function transforms an array A into "the permutation next to A in the lexicographic order." We give some examples.

- `std::next_permutation` transforms the array $(2, 1, 3)$ into $(2, 3, 1)$.
- `std::next_permutation` transforms the array $(1, 4, 6, 6, 2)$ into $(1, 6, 2, 4, 6)$.
- `std::next_permutation` transforms the array $(1, 2, 2, 2, 1, 1)$ into $(2, 1, 1, 1, 2, 2)$.

Thus, if we apply `std::next_permutation` to $(1, 2, 3)$ five times, the array is transformed as follows: $(1, 2, 3) \to (1, 3, 2) \to (2, 1, 3) \to (2, 3, 1) \to (3, 1, 2) \to (3, 2, 1)$. This way, we can search all the possibilities. Note that `std::next_permutation` returns `false` only if there is no next permutation. Otherwise, it returns `true`. We can search all the permutations of $1, \ldots, N$ by the following way.

```
vector<int> perm(N);
for (int i = 0; i < N; i++) {
    perm[i] = i + 1; // Initially, set perm = (1, 2, ..., N)
}
```

[*1] A permutation means an arrangement of $(1, 2, \ldots, N)$. In a competitive programming contest, tasks calculating all the possible "permutations" like arrangements of the giraffes appear frequently.

```
do {
    // In this loop, do some calculation for the permutation perm
} while(next_permutation(perm.begin(), perm.end()));
```

By a triple loop for possible values of (i, j, k_1) and (i, j, k_2), we can determine whether each permutation satisfies the condition of the visual quality or not. The time complexity is $O(N^3)$. Therefore, we can obtain the answer in $O(N! \times N^3)$ time. We can solve Subtask 1. Here is a sample implementation.

```
#include <vector>
#include <iostream>
#include <algorithm>
using namespace std;

bool check(int N, const vector<int>& P) {
    // This function determines whether the arrangement P[0], P[1], ..., P[N-1]
    // of the N giraffes satisfies the condition or not.
    for (int l = 0; l < N; l++) {
        for (int r = l + 1; r < N; r++) {
            bool flag1 = false, flag2 = false;
            for (int i = l + 1; i < r; i++) {
                if (perm[l] < perm[i] && perm[i] > perm[r]) {
                    flag1 = true;
                }
                if (perm[l] > perm[i] && perm[i] < perm[r]) {
                    flag2 = true;
                }
            }
            if (flag1 == true && flag2 == true) {
                return false; // l, r with bad visual quality are found.
            }
        }
    }
    return true; // There is no l, r with bad visual quality.
}

```

```cpp
int main() {
    // Input
    int N;
    cin >> N;
    vector<int> P(N);
    for (int i = 0; i < N; i++) {
        cin >> P[i];
    }

    // Calculate the answer
    int answer = 1000000000;
    vector<int> perm(N);
    for (int i = 0; i < N; i++) {
        perm[i] = i + 1;
    }
    do {
        if (check(N, P) == true) {
            // Calculate unmatch, the number of moved giraffes.
            int unmatch = 0;
            for (int i = 0; i < N; i++) {
                if (perm[i] != P[i]) {
                    unmatch += 1;
                }
            }
            answer = min(answer, score);
        }
    } while(next_permutation(perm.begin(), perm.end()));

    // Output the answer
    cout << answer << endl;

    return 0;
}
```

3 Subtask 2 ($N \leq 13$)

In Subtask 2 and later subtasks, a "permutation" means a permutation of $(1, 2, \ldots, N)$ if not otherwise specified.

First, when we solve such a task, it is important to think of an easy **characterization**. For this problem, we should think about "what are the permutations satisfying the condition of visual quality?" The reason is, in its original form, the condition in the task statement is too complicated to obtain efficient solution. To start, let us consider which permutations P satisfy the condition, and which permutations P do not satisfy the condition.

For example, when $N = 5$, which permutations P satisfy the condition? Observing a situation well, you may notice that one of the ends is either 1 or 5. The reason for this is that if both of the ends are $2, 3, 4$, the visual quality becomes bad when we take a picture with $l = 1, r = 5$. In general, the following holds.

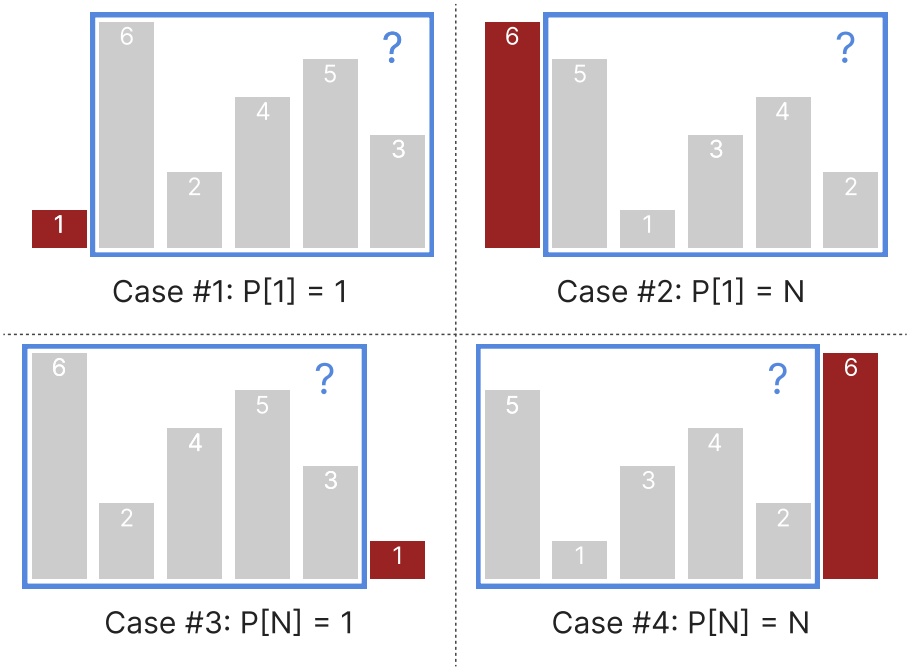
Property 1 If both P_1 and P_N are between 2 and $N - 1$, inclusive, then the permutation P does not satisfy "the condition of the visual quality."

The reason is that if both P_1 and P_N are between 2 and $N - 1$, inclusive, then the following hold:

- One of $P_2, \ldots, P_{N-1}$ is equal to 1 (we write it as P_{k_1}). Then we have $P_1 > P_{k_1} < P_N$.
- One of $P_2, \ldots, P_{N-1}$ is equal to N (we write it as P_{k_2}). Then we have $P_1 < P_{k_2} > P_N$.

Therefore, the visual quality becomes bad if we choose $l = 1, r = N$. Conversely, we obtain a necessary condition for "the condition of the visual quality" to hold.

Property 2 For the permutation P to satisfy "the condition of the visual quality," it is necessary that at least one of $P_1 = 1, P_1 = N, P_N = 1, P_N = N$ holds.



If $P_1 = N$, the remaining part $(P_2, P_3, \ldots, P_N)$ of the permutation (it is a permutation of $(2, 3, \ldots, N)$) has to satisfy Property 2. Thus, if we choose one of the ends among the four possibilities, the remaining part (surrounded by a blue box) of length $N - 1$ in the above figures has to satisfy Property 2 too. Similarly, for each of the four possibilities, the remaining part of length $N - 2$ has to satisfy Property 2. The same is true for the remaining part of length $N - 3, \ldots$ etc. Therefore, for "the condition of the visual quality" to hold, it is necessary that the permutation P satisfies the following condition.

Property 3 For "the condition of the visual quality" to hold, it is necessary that the permutation P is generated in the following way.

1. In the beginning, we set $l = 1, r = N, d = 1, u = N$. Initially, none of $P_1, P_2, \ldots, P_N$ is determined.
2. Repeat the following N times.
   - Choose one of the following operations (a)-(d), and perform it.
     (a) Set $P_l = d$, increase l by 1, and increase d by 1.
     (b) Set $P_l = u$, increase l by 1, and decrease u by 1.
     (c) Set $P_r = d$, decrease r by 1, and increase d by 1.
     (d) Set $P_r = u$, decrease r by 1, and decrease u by 1.

At most 4^N permutations P are generated by Property 3[*2]. We see this from a tree diagram since we have 4 possibilities for each of N steps. Moreover, we see that the permutations generated in the above way always

[*2] Of course, some permutations appear more than once.

satisfy the condition.

Property 4 Every permutation P generated by Property 3 satisfies "the condition of the visual quality."

Proof It is enough to prove that the visual quality does not become bad for any choice of l, r $(1 \leq l \leq r \leq N)$. Among P_l and P_r, let P_{first} be the element which is chosen first.

- If P_{first} is chosen by (a) or (c), the elements $P_{l+1}, \ldots, P_{r-1}$ chosen later than P_{first} are larger than the value of d $(= P_{first})$ when P_{first} is chosen. Thus there does not exist k $(l < k < r)$ satisfying $P_l > P_k < P_r$. The visual quality does not become bad.
- If P_{first} is chosen by (b) or (d), the elements $P_{l+1}, \ldots, P_{r-1}$ chosen later than P_{first} are smaller than the value of u $(= P_{first})$ when P_{first} is chosen. Thus there does not exist k $(l < k < r)$ satisfying $P_l < P_k > P_r$. The visual quality does not become bad.

Since the visual quality does not become bad in every case, the permutation satisfies the condition.

If we generate 4^N permutations by Property 3, we obtain all the permutations satisfying the condition. Moreover, all the permutations satisfying the condition can be obtained in this way. Among them, the minimum number of "the elements which are changed from the initial permutation" is the answer. Thus we can solve this task in $O(4^N \times N)$ time in total. We can solve Subtask 2 in this way. In the following, we give a sample implementation using recursive function calls[*3].

```
#include <vector>
#include <iostream>
#include <algorithm>
using namespace std;

int brute_force(int N, const vector<int>& P, \
                int l, int r, int d, int u, vector<int>& perm) {
    if (r - l == -1) {
        // N operations are finished.
        int score = 0;
        for (int i = 0; i < N; i++) {
            if (perm[i] != P[i]) {
                score += 1;
            }
        }
```

[*3] A recursive function call means a function call which calls itself. If you want to learn about it, please search the websites.

```
16          return score;
17      }
18      // Operation (a)
19      perm[l] = d;
20      int res1 = brute_force(N, P, l + 1, r, d + 1, u, perm);
21      perm[l] = -1;
22      // Operation (b)
23      perm[l] = u;
24      int res2 = brute_force(N, P, l + 1, r, d, u - 1, perm);
25      perm[l] = -1;
26      // Operation (c)
27      perm[r] = d;
28      int res3 = brute_force(N, P, l, r - 1, d + 1, u, perm);
29      perm[r] = -1;
30      // Operation (c)
31      perm[r] = u;
32      int res4 = brute_force(N, P, l, r - 1, d, u - 1, perm);
33      perm[r] = -1;
34      // Return the answer
35      return min(min(res1, res2), min(res3, res4));
36  }
37
38  int main() {
39      // Input
40      int N;
41      cin >> N;
42      vector<int> P(N);
43      for (int i = 0; i < N; i++) {
44          cin >> P[i];
45          P[i] -= 1; // P[i] begins from 0
46      }
47
48      // Calculate the answer -> Output it
49      // Note that we implemented so that the indices and P[i]
```

```
50      // begin from 0. (It is different from the text of the review.
51      // In the review, the indices begin from 1.)
52      // Thus the initial status is (l, r, u, d) = (0, N-1, 0, N-1)
53      vector<int> perm(N, -1);
54      int answer = brute_force(N, P, 0, N - 1, 0, N - 1, perm);
55      cout << answer << endl;
56
57      return 0;
58  }
```

Incidentally, by the algorithm explained as above, the same permutation may appear several times. In fact, the number of permutations satisfying the condition is much smaller than 4^N; there are only 0.147×3.42^N permutations satisfying it[*4]. If we implement the algorithm appropriately so that the same permutation does not appear more than once, we can solve this task in $O(3.42^N)$ time. For each input data of Subtask 2, we can calculate the answer within 0.01 seconds.

4 Subtask 3 ($N \leq 300$)

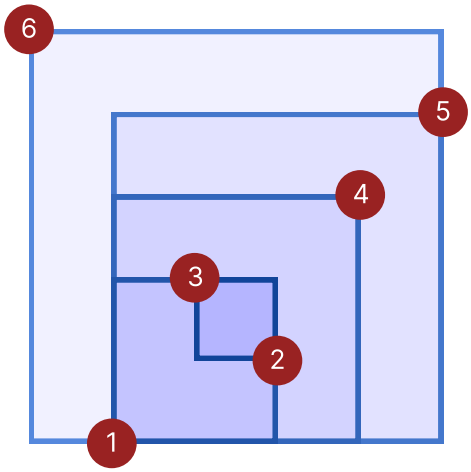
Using ideas of the solution of Subtask 2, we can solve this task in $O(N^3)$ time by Dynamic Programming (DP)[*5]. Since many tasks can be solved by DP in Informatics Olympiads, you are encouraged to learn about it.

First, in order to understand the solutions graphically, we draw a 2-dimensional picture describing the setting of the task[*6]. Let the x-coordinates be the indices i (= the positions of the giraffes) of the permutation, and the y-coordinates be the values P_i (= the heights of the giraffes) of the permutation. We draw a picture representing the generation of permutations in Property 3. Then we have the following stratum structure like a Baumkuchen. (The following picture describes the generation of the permutation $(P_1, P_2, P_3, P_4, P_5, P_6) = (6, 1, 3, 2, 4, 5)$ for $N = 6$.)

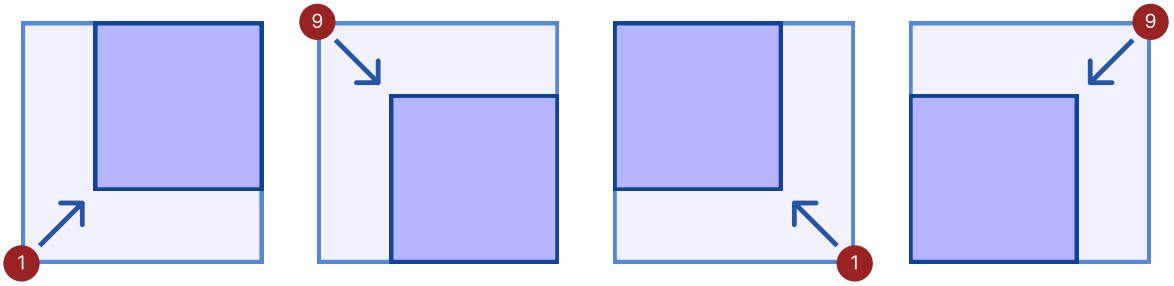
[*4] Let a_N be the number of permutations of length N satisfying the condition. We have the recurrence formula $a_1 = 1, a_2 = 2, a_k = 4a_{k-1} - 2a_{k-2}$. Here, we subtract $2a_{k-2}$ because if "$P_1 = 1$ and $P_N = N$ are satisfied" and "$P_1 = N$ and $P_N = 1$ are satisfied," the same permutation is counted twice. Calculating the general term of the sequence, we obtain $a_N = \frac{2-\sqrt{2}}{4}\left((2+\sqrt{2})^n + (3+2\sqrt{2})(2-\sqrt{2})^n\right)$.

[*5] In Dynamic Programming (DP), we decompose the problem into smaller problems, and calculate the answers recursively from smaller problems to larger ones. If you do not know about it, please search the websites.

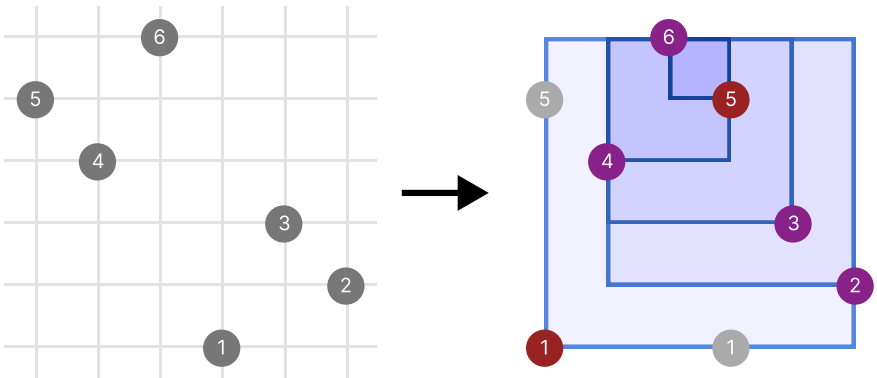
[*6] In some tasks, we can easily understand the solutions when we draw a picture in the 2-dimensional plane.



In this picture the operation in Property 3 corresponds to the following operation "choose and 'fix' one of the four vertices of the square, and shrink the side length of the square by 1 toward the opposite direction."



Namely, the task can be rephrased as "when we make a stratum structure as above, what is the maximum number of points which coincide with one of $(1, P_1)$, $(2, P_2)$, ..., (N, P_N)?" (The following picture corresponds to the permutation $(P_1, P_2, P_3, P_4, P_5, P_6) = (5, 4, 6, 1, 3, 2)$ in Sample Input 1. If we generate the permutation $(1, 4, 6, 5, 3, 2)$ as in the right figure, 4 points coincide. Since this is the maximum value, the answer is $6-4 = 2$.)



Then we perform DP using this idea. We shall calculate the following values $dp[l][x][y]$ from smaller values of l.

- $dp[l][x][y]$ = (the maximum number of points which coincide with the initial points $(1, P_1)$, $(2, P_2)$, ..., (N, P_N) inside the square whose lower-left vertex is (x, y) and side length is l)

There are four possible choice to shrink the side length of the square by 1. For each choice, from the state of $dp[l][x][y]$ ($l \geq 1$, $1 \leq x \leq N - l$, $1 \leq y \leq N - l$), we can calculate "the maximum number of points which coincide with the initial points inside $dp[l][x][y]$" as follows.

- If we choose (a), the number of points which coincide is

$$dp[l-1][x+1][y+1] + \begin{cases} 1 & P_x = y \\ 0 & \text{otherwise.} \end{cases}$$

- If we choose (b), the number of points which coincide is

$$dp[l-1][x+1][y] + \begin{cases} 1 & P_x = y + l \\ 0 & \text{otherwise.} \end{cases}$$

- If we choose (c), the number of points which coincide is

$$dp[l-1][x][y+1] + \begin{cases} 1 & P_{x+l} = y \\ 0 & \text{otherwise.} \end{cases}$$

- If we choose (d), the number of points which coincide is

$$dp[l-1][x][y] + \begin{cases} 1 & P_{x+l} = y + l \\ 0 & \text{otherwise.} \end{cases}$$

The value of $dp[l][x][y]$ is the maximum of the above 4 values. Thus, if we use the results for smaller problems, we can immediately calculate the values of $dp[l][x][y]$.

Since the maximum number of points which coincide is $dp[N-1][1][1]$, the answer to this task is $N - dp[N-1][1][1]$. We can solve this task in $O(N^3)$ time in total. We can solve Subtask 3 in this way. Here is a sample implementation.

```
#include <vector>
#include <iostream>
#include <algorithm>
using namespace std;

int solve(int N, const vector<int>& P) {
```

```
7       // Calculate the answer by DP
8       // Note that we implemented so that the indices and P[i]
9       // begin from 0. (It is different from the text of the review.
10      // In the review, the indices begin from 1.)
11      // x, y in dp[l][x][y] begin from 0.
12      vector<vector<vector<int> > > dp(N);
13
14      // Calculate the initial DP values (when l = 0)
15      dp[0] = vector<vector<int> >(N, vector<int>(N));
16      for (int x = 0; x < N; x++) {
17          for (int y = 0; y < N; y++) {
18              dp[0][x][y] = (P[x] == y ? 1 : 0);
19          }
20      }
21
22      // Calculate the DP values for l = 1, 2, ..., N-1 in this order.
23      for (int l = 1; l <= N - 1; l++) {
24          dp[l] = vector<vector<int> >(N - l, vector<int>(N - l));
25          for (int x = 0; x < N - i; x++) {
26              for (int y = 0; y < N - i; y++) {
27                  int v1 = dp[l - 1][x + 1][y + 1] + (P[x] == y ? 1 : 0);
28                  int v2 = dp[l - 1][x + 1][y] + (P[x] == y + l ? 1 : 0);
29                  int v3 = dp[l - 1][x][y + 1] + (P[x + l] == y ? 1 : 0);
30                  int v4 = dp[l - 1][x][y] + (P[x + l] == y + l ? 1 : 0);
31                  dp[l][x][y] = max(max(v1, v2), max(v3, v4));
32              }
33          }
34      }
35
36      // Return the answer
37      // Note that we implemented the solution so that x, y begin from 0.
38      return N - dp[N - 1][0][0];
39  }
40
```

```
int main() {
    // Input
    int N;
    cin >> N;
    vector<int> P(N);
    for (int i = 0; i < N; i++) {
        cin >> P[i];
        P[i] -= 1; // P[i] begins from 0.
    }

    // Calculate the answer -> Output it
    int answer = solve(N, P);
    cout << answer << endl;

    return 0;
}
```

Actually, we can improve the constant in the $O(N^3)$ solution drastically as follows. We use the following two techniques.

- DP reusing memories.
- Optimization using `#pragma`.

We explain the first technique. The original implementation uses a 3-dimensional array of $dp[l][x][y]$. When N becomes large, it uses large amount of memories. It also slows down the execution of the program because we cannot use cache memories efficiently. For the DP in this case, we can implement it reusing a 2-dimensional array as follows [*7]. Please see the difference from the previous implementation.

```
int solve(int N, const vector<int>& P) {
    vector<vector<int> > dp(N, vector<int>(N));
    for (int x = 0; x < N; x++) {
        for (int y = 0; y < N; y++) {
            dp[x][y] = (P[x] == y ? 1 : 0);
        }
```

[*7] Generally speaking, when we implement a DP which reuses memories, we have to be careful about the order to calculate (update) the DP values. In this case, we cannot obtain a correct answer if we update them from larger values of x, y.

```
    }
    for (int i = 1; i <= N - 1; i++) {
        for (int x = 0; x < N - i; x++) {
            for (int y = 0; y < N - i; y++) {
                int v1 = dp[x + 1][y + 1] + (P[x] == y ? 1 : 0);
                int v2 = dp[x + 1][y] + (P[x] == y + i ? 1 : 0);
                int v3 = dp[x][y + 1] + (P[x + i] == y ? 1 : 0);
                int v4 = dp[x][y] + (P[x + i] == y + i ? 1 : 0);
                dp[x][y] = max(max(v1, v2), max(v3, v4));
            }
        }
    }
    return N - dp[0][0];
}
```

Next, we explain the latter technique. Perhaps you might be surprised if you haven't heard about it before. The execution of the program can be drastically speeded up if you add the following two lines in the beginnig of the source code.

```
#pragma GCC target("avx2")
#pragma GCC optimize("O3")
```

By these two lines, the auto-vectorization of GCC is enabled, and a part of the program is compiled using fast instructions of SIMD. SIMD provides an instruction to perform a 128 or 256 bit calculation simultaneously. It can be used in most PC's today[*8]. Using this, for example, we can perform 8 addition operations of 32 bit integers simultaneously, and it is expected that the execution becomes 4 times faster than before[*9].

Since the above DP program is rather simple, we can expect speeding-up by SIMD. In fact, adding the above two lines, we can calculate the answer within about 1 second when $N = 2000$.

[*8] Of course, it can be used in most programming contests. If we cannot use AVX2, we can sometimes use AVX or SSE4 to speed up the program.

[*9] If you want to learn about SIMD (Single Instruction Multiple Data), please search the websites. For a reference, see `https://yukicoder.me/wiki/auto_vectorization` (in Japanese).

5 How to "use the randomness of input data"

Up to this point, we have never used the special condition that "$P_1, P_2, \ldots, P_N$ in input data are generated randomly." To obtain a full score solution, we essentially use this condition.

First of all, if the input data is random, what is the difference from the general case. Generally speaking, some of the input data are unbiased, and some of the input data are "spite." If the randomness is guaranteed, we do not have to consider the biased data. Using this, we can sometimes reduce the computational complexity. Perhaps some of you see such kind of tasks for the first time, we give an example.

Problem Given a sequence $a = (a_1, a_2, \ldots, a_N)$ of N real numbers, write a program which sorts it in ascending order[*10]. Here the values a_i $(1 \leq i \leq n)$ are greater than or equal to 0 and strictly less than 1, and these values are generated by uniform random numbers.

If you have some knowledge on algorithms, you may know this problem can be solved in $O(n \log n)$ time. However, if we use the fact that the inputs are random, we can solve it in $O(n)$ time on average as follows.

1. For $k = 0, 1, \ldots, n-1$, we define an array v_k to be "the array consisting of all a_i $(1 \leq i \leq n)$ which are larger than or equal to k/n and strictly less than $(k+1)/n$."
2. We sort each of the n arrays $v_0, v_1, \ldots, v_{n-1}$ in ascending order.
3. Combining the n sorted sequences $v_0, v_1, \ldots, v_{n-1}$ from the beginning successively, we obtain a sorted sequence.

[*10] This means rearranging the sequence from smaller values. For example, if the sequence $(0.7, 0.4, 0.9, 0.6, 0.1)$ is given, we obtain the sequence $(0.1, 0.4, 0.6, 0.7, 0.9)$ after sorting it in ascending order.

If we consider general inputs, for example, if the input is $a = (0, 1/n^2, 2/n^2, \ldots, (n-1)/n^2)$, then all the values a_i come to the array v_0, and sorting takes time after all. However, if the inputs are random (if you are not too unlucky), this cannot happen. Since the expected value of $|v_i|^2$ is $2 - 1/n$, even if sorting each v_i takes $O(|v_i|^2)$ time, it is solved in $O(n)$ time in total[*11]. This example show that we can sometimes speed up the computational complexity of the algorithm using "the unbiasedness" of input data.

Now we return to the main theme of this section. In the sorting problem above, a nice property is that "the expected value of $|v_0|^2 + |v_1|^2 + \cdots + |v_{n-1}|^2$ is small" for random inputs, and we use this property to achieve the time complexity $O(n)$. In the task of giraffes, what are the nice properties satisfied for random inputs? The answer is that, in the average cases, the maximum number of unmoved giraffes (= N − (answer)) is $O(\sqrt{N})$.

Intuitively, you may wonder that most giraffes have to move, but it is not the case. Let us consider the reason. First, since we have "a stratum structure" as in Subtask 3, we see that any permutation $P = (P_1, P_2, \ldots, P_N)$ satisfying "the condition of the visual quality" can be decomposed into an increasing sequence $(P_{s_1}, P_{s_2}, \ldots, P_{s_k})$ and a decreasing sequence $(P_{t_1}, P_{t_2}, \ldots, P_{t_{N-k}})$. Too see this, when we shrink the side length of the square by 1, we put the element into the sequence A if we choose the lower-left or the upper-right vertex, and the sequence B if we choose the upper-left or the lower-right vertex. The sequence A is always increasing and the sequence B is always decreasing. Thus we have the following conclusion.

Property 5 (the maximum number of unmoved giraffes) ≤ (longest increasing subsequence of P) + (longest decreasing subsequence of P).

Therefore, if the length of longest increasing subsequences (LIS) and longest decreasing subsequences are sufficiently small in the average cases, the number of unmoved giraffes is sufficiently small. In fact, it is true, and the following is known.

Property 6 The expected value of the length of LIS of a randomly chosen permutation $p = (p_1, p_2, \ldots, p_n)$ of length n is approximately equal to $2\sqrt{n}$.

More precisely, it is known that the expected value of the length of LIS is $2\sqrt{n} + O(n^{1/6})$, and the standard deviation is $O(n^{1/6})$ (Odlyzko & Rains, 1998). Perhaps, it may sound difficult. But we can show the following related result relatively easily.

[*11] The reason why the expected value of $|v_i|^2$ is equal to $2 - 1/n$ is seen as follows. For $j = 1, 2, \ldots, n$, we put $x_j = 1$ if a_j is contained in v_i, and $x_j = 0$ otherwise. Then, we have $|v_i|^2 = (x_1 + x_2 + \cdots + x_n)^2 = (x_1^2 + x_2^2 + \cdots + x_n^2) + 2(x_1x_2 + x_1x_3 + \cdots + x_{n-1}x_n)$. Since the expected value of x_j^2 is $1/n$ (the probability that a_j is contained in v_i is $1/n$), and the expected value of x_jx_k $(j \neq k)$ is $1/n^2$ (the probability that both a_j, a_k are contained in v_i is $1/n^2$), the expected value is the sum is $n \cdot 1/n + 2 \cdot (n(n-1)/2 \cdot 1/n^2) = 2 - 1/n$.

We consider the expected value of the number of increasing subsequences of length k. There are $\binom{n}{k}$ ways to choose a subsequence of length k. The probability that the chosen subsequence is increasing is $1/k!$. Therefore, when $k \ll n$, the expected value of the number of increasing subsequences of length k is approximately equal to[a]

$$\binom{n}{k} \times \frac{1}{k!} = \frac{n(n-1)\cdots(n-k+1)}{(k!)^2} \approx \frac{n^k}{(k!)^2} \approx n^k \cdot \left(\sqrt{2\pi k}\left(\frac{k}{e}\right)^k\right)^{-2} = 2\pi k \cdot \left(\frac{ne^2}{k^2}\right)^k.$$

Therefore, when $k \approx e\sqrt{n}$ holds[b], the expected value of the number of increasing subsequences of length k becomes 1. If k is larger than it, the probability that there exists an increasing subsequences of length k tends to 0 indefinitely.

[a] We use Stirling's formula $k! \approx \sqrt{2\pi k}(k/e)^k$ to estimate the value. Here $e = 2.71828\ldots$ is the base of the natural logarithms.
[b] When $k = (e + o(1))\sqrt{n}$, precisely.

Combining these two properties, we have the following fact.

Property 7 In the average cases, the maximum number of unmoved giraffes is approximately less than $4\sqrt{N}$.

Moreover, we experimentally know that this value is approximately equal to $2.8\sqrt{N}$. We reduce the computational complexity using this fact. It is explained in the following sections.

6 A solution of computational complexity $O(N^{2.5})$

In the above discussions, we see that the maximum number of points which coincide with the initial points is approximately less than $2.8\sqrt{N}$. Here we explain how to use it to reduce the computational complexity.

As a first step, we recall the Knapsack problem, which is a famous problem for DP.

0-1 Knapsack Problem There are n items. The i-th item ($1 \leq i \leq n$) has value v_i and weight w_i. When we choose items so that the total weight does not exceed W, what is the maximum total value?

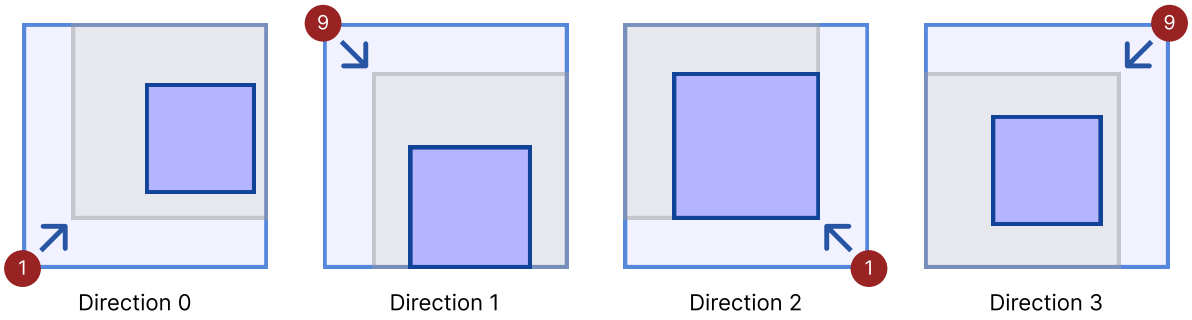
When the constraints on w_i are small, this problem can be solved by DP in $O(N^2 \cdot \max w_i)$ time by putting $dp[i][j]$ = (the maximum total value if we fix the first i items whose total weight is j). On the other hand, when the constraints on w_i are large, if the constraints on v_i are small, we have a similarly efficient solution. It can be solved by DP in $O(N^2 \cdot \max v_i)$ time by putting $dp[i][j]$ = (the minimum total weight if we fix the first i items whose total value is j).

The current task has the property that "the side length of a square" which corresponds to the weight in the knapsack problem is large, whereas "the maximum number of points which coincide with the initial points" which corresponds to the value in the knapsack problem is small. Thus it is natural to expect that the computa-

tional complexity may be reduced if we use a solution similar to the knapsack problem for small v_i. We consider a DP calculating the following values.

- $dp[i][j][k]$ = (the minimum side length of a square if at most k points in the square with vertex (i, P_i) of direction j ($j = 0, 1, 2, 3$) coincide with the initial points $(1, P_1), (2, P_2), \ldots, (N, P_N)$)

Although (x, y) is used to represent a state in the original DP, we use (i, j) here. The reason is that we need the information of the directions because there are 4 directions to increase the value of l, and the DP value is increased by 1 only if, for some i, the point (i, P_i) is contained as the vertex of direction i. We can combine a number of state transitions which do not change the DP values into a single state transition. Thus, as in the following figure, we can perform a state transition if the square contains a smaller square.



Now we have an idea of the solution. We shall consider how to perform a DP calculation. First, "the square described by $dp[i][j][k]$" is given as follows.

- The square described by $dp[i][0][k]$: $i \leq x \leq i + dp[i][0][k]$, $P_i \leq y \leq P_i + dp[i][0][k]$
- The square described by $dp[i][1][k]$: $i \leq x \leq i + dp[i][1][k]$, $P_i - dp[i][1][k] \leq y \leq P_i$
- The square described by $dp[i][2][k]$: $i - dp[i][2][k] \leq x \leq i$, $P_i \leq y \leq P_i + dp[i][2][k]$
- The square described by $dp[i][3][k]$: $i - dp[i][3][k] \leq x \leq i$, $P_i - dp[i][3][k] \leq y \leq P_i$

Thus, for each k, we have a square for each (i, j). In total, we have at most $4N$ squares. We calculate $dp[i][j][k + 1]$ using the $4N$ squares as follows.

- $dp[i][0][k + 1]$ = (the minimum value of l so that the square region $i + 1 \leq x \leq i + l$, $P_i + 1 \leq y \leq P_i + l$ contains one of the $4N$ squares entirely)
- $dp[i][1][k + 1]$ = (the minimum value of l so that the square region $i + 1 \leq x \leq i + l$, $P_i - l \leq y \leq P_i - 1$ contains one of the $4N$ squares entirely)
- $dp[i][2][k + 1]$ = (the minimum value of l so that the square region $i - l \leq x \leq i - 1$, $P_i + 1 \leq y \leq P_i + l$ contains one of the $4N$ squares entirely)
- $dp[i][3][k + 1]$ = (the minimum value of l so that the square region $i - l \leq x \leq i - 1$, $P_i - l \leq y \leq P_i - 1$

contains one of the $4N$ squares entirely)

Each $dp[i][j][k]$ can be calculated in $O(N)$ time. Since there are approximately $4N^2$ combinations of the triples (i, j, k), you may wonder if the computational complexity is $O(N^3)$. However, in the average cases, the number of points which coincide with the initial points is approximately less than $2.8\sqrt{N}$. Thus, if we quit the loop when no square remains, we need to calculate the DP values for $k \leq 2.8\sqrt{N}$ only. Therefore, in the average cases, this task can be solved in $O(N^{2.5})$ time. We give a sample implementation below. Note that since we store the DP values in an 2-dimensional array, the value `dp[i][j]` corresponds to $dp[i][j][k]$, and the value `ndp[i][j]` corresponds to $dp[i][j][k+1]$. In the source code, we omit the part of inputs and outputs, and `#include`, etc.

```
class square {
public:
    int x, y, l;
    square() : x(0), y(0), l(0) {};
    square(int x_, int y_, int l_) : x(x_), y(y_), l(l_) {};
};

int solve(int N, const vector<int>& P) {
    const int INF = 1012345678;
    vector<vector<int> > dp(4, vector<int>(N, 0));
    int t = 1;
    while (true) {
        vector<square> sq;
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < N; j++) {
                if (dp[i][j] != INF) {
                    int x = j - (i & 2 ? dp[i][j] : 0);
                    int y = P[j] - (i & 1 ? dp[i][j] : 0);
                    sq.push_back(square(x, y, dp[i][j]));
                }
            }
        }
        int S = sq.size();
        dp = vector<vector<int> >(4, vector<int>(N, INF));
```

```
            for (int i = 0; i < 4; i++) {
                for (int j = 0; j < N; j++) {
                    int minl = INF;
                    for (int k = 0; k < S; k++) {
                        int xdiff = (i & 2 ? j - (sq[k].x + sq[k].l) : sq[k].x - j);
                        int ydiff = (i & 1 ? P[j] - (sq[k].y + sq[k].l) : sq[k].y - P[j]);
                        if (xdiff > 0 && ydiff > 0) {
                            minl = min(minl, max(xdiff, ydiff) + sq[k].l);
                        }
                    }
                    int xlim = (i & 2 ? j : (N - 1) - j);
                    int ylim = (i & 1 ? P[j] : (N - 1) - P[j]);
                    if (minl <= min(xlim, ylim)) {
                        dp[i][j] = minl;
                    }
                }
            }
            if (dp == vector<vector<int> >(4, vector<int>(N, INF))) {
                break;
            }
            t += 1;
        }
        return N - t;
    }
```

This solution has a large constant compared with other $O(N^3)$ solutions by DP. We cannot expect speeding-up by SIMD. When the size of the input data is about $N = 2000$, the actual execution is slower than the optimized $O(N^3)$ solution. However, as explained in the next section, we can speed up this solution using data structures, which leads to a full score solution.

7 Full score solution ($N \leq 8000$)

In the previous solution, we calculate $O(N^{1.5})$ DP values. Since each value is calculated in $O(N)$ time, the total computational complexity is $O(N^{2.5})$. We shall speed up it using data structures. We consider how to calculate each $dp[i][j][k]$ in $O(\log N)$ time, for example.

To do this, we need to treat the following problem. Since it is slightly complicated, we recommend you draw figures on paper.

Problem There are n squares. The lower-left vertex of the square i ($1 \leq i \leq n$) is (x_i, y_i), and the upper-right vertex of the square i is $(x_i + l_i, y_i + l_i)$.We need to answer the following Q queries. The j-th query ($1 \leq j \leq Q$) is as follows. (It is allowed to read the queries beforehand.)

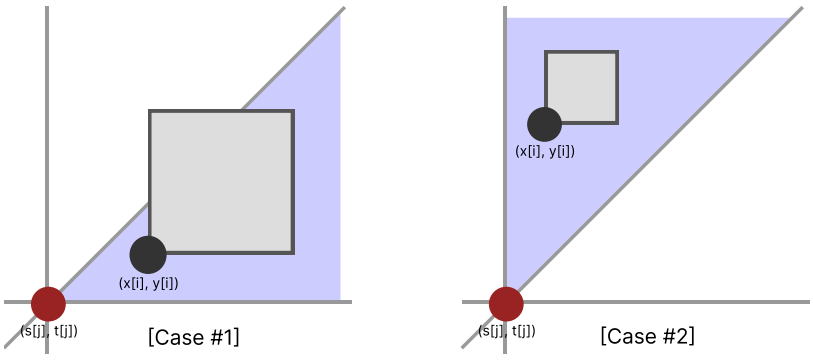
- Given s_j, t_j, what is the minimum value of l so that the square region $s_j \leq x \leq s_j + l,\ t_j \leq y \leq t_j + l$ contains at least one of the n squares entirely?

The constraints of the problem is that the square regions and the points (s_j, t_j) given by the queries are entirely contained in the region $0 \leq x \leq n$, $0 \leq y \leq n$.

Here we consider the problem only for the direction $j = 0$. The cases of the directions $j = 1, 2, 3$ can be treated similarly by rotation.

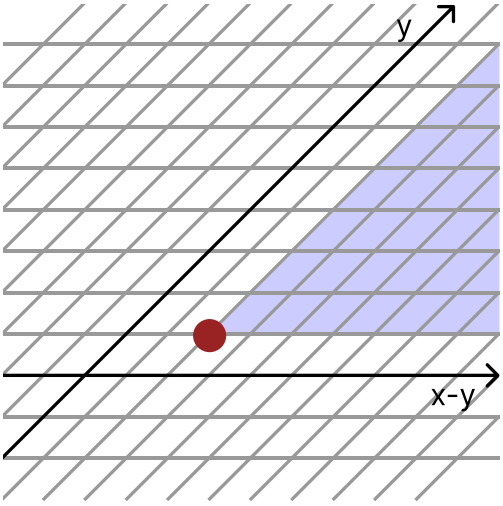
We shall show this problem can be solved in $O((N+Q)\log N)$ time by a plane sweep algorithm and a segment tree. To see this, we summarize the situation of the problem. The answer to each query can be calculated in the following way.

- The minimum of $\max(x_i - s_j, y_i - t_j) + l_j$ for i satisfying $x_i \geq s_j$, $y_i \geq t_j$.
- Namely, the smaller of the following two values.
  1. The minimum of $x_i - s_j + l_j$ for i satisfying $y_i \geq t_j$, $x_i - y_i \geq s_j - t_j$. (In Case #1, the gray point is contained in the blue region.)
  2. The minimum of $y_i - t_j + l_j$ for i satisfying $x_i \geq s_j$, $x_i - y_i \leq s_j - t_j$. (In Case #2, the gray point is contained in the blue region.)



Perhaps advanced contestants may notice that we can use a plane sweep algorithm to solve these problems. A key to a plane sweep algorithm is that the value of the query j and the value of the data i are independenct. We achieve this by decomposing the original problem into two sub-problems.

Now we shall solve Problem 1 by a plane sweep algorithm. We can also solve Problem 2 similarly since the shape of the region is almost the same. Note that the region is given in terms of y and $x - y$. It is easier to consider the $(y, x - y)$-coordinates than the usual (x, y)-coordinates.



The rest is a typical plane sweep problem. Let the sweeping line be $x - y = k$. We sweep the plane by decreasing the value of k. (In the above figure, we move the line from right to left.) Then the answer to the question is calculated as "the minimum of x_i among the currently added points whose y coordinate is larger than or equal to t_j." This value can be quickly calculated by an RMQ[*12] segment tree which records the current minimum value of x_i for each y-coordinate. More precisely, the algorithm is described as follows.

1. We give an array $a = (a_0, a_1, \ldots, a_{n-1})$ of length n. Let the initial values be ∞.
2. From the larger values of $x_j - y_j$ and $s_j - t_j$, we sort the $N + Q$ points (x_i, y_i) and (s_j, t_j) together. If two points are the same, we assume that (x_i, y_i) comes before (s_j, t_j)[*13].
3. For $i = 1, 2, \ldots, N + Q$, in this order, we perform the following operations.
   - If the i-th point in the sorted sequence is (x_i, y_i), we let a_{y_i} be $\min(a_{y_i}, x_i)$.
   - If the i-th point in the sorted sequence is (s_j, t_j), the answer to the query j is $\min(a_0, a_1, \ldots, a_{t_j}) - s_j + l_j$.

Here the operation 3 is "to update a point, and obtain the minimum value on the interval." We can perform it in $O(\log N)$ using a segment tree. Sweeping the plane and calculating the answers at once, we solve Problem 1 in $O((N + Q) \log N)$ time.

Similarly, we can solve Problem 2 in $O((N + Q) \log N)$ time. In total, we can solve Problem on page 21 in

[*12] The abbreviation of Range Minimum Query. Here we consider a query to update a point, and obtain the minimum value on the interval. We can perform each of these queries in $O(N \log N)$ time.

[*13] It takes $O((N + Q) \log(N + Q))$ time if we use `std::sort`. If we sort them by Counting Sort, it can be done in $O(N + Q)$ time.

$O((N+Q)\log N)$ time. Therefore, we can calculate the values of $dp[i][j][k+1]$ for all (i, j) in $O(N \log N)$ time. This way, we can solve every test case of this task in $O(N^2 \log N)$ time.

As explained before, since the size of k becomes at most $2.8\sqrt{N}$, the time complexity is $O(N^{1.5} \log N)$ on average, and we get the full score.

A sample implementation of the full score solution can be downloaded from the contest website. If we rotate the situation appropriately, we do not need a source code for each of $j = 0, 1, 2, 3$. Moreover, we can reduce Problem 2 to Problem 1. Thus we need only implement a plane sweep algorithm in one subroutine actually. We can devise the implementation of the solution to decrease the possibility of bugs in the program.

Finally, to implement a plane sweep algorithm, we can use a data structure similar to Binary Indexed Tree (BIT)[*14] instead of a segment tree. Perhaps you may have an impression that BIT operates queries to calculate sums. In this task, there are properties such as "the values a_i of the array keeps decreasing" and "to calculate the minimum of $a_0, a_1, \ldots, a_k$, not the sum of them." We can use a data structure similar to BIT to manage the minimum values. This way, we can slightly speed up the program.

8 Conjecture: Can we reduce the computational complexity further?

Some of you might have noticed that if the input data is random, a used point (i, P_i) tends to get closer to the line $y = x$ or the line $y = N - x$.

This is true also for a LIS in a random permutation. We can prove[*15] that the distance between any point (i, p_i) in a LIS and the line $y = x$ is at most $O(n^{7/8})$. We conjecture similar property holds for this task.

Conjecture 1 The distance between a used point and the line $y = x$ or the line $y = N - x$ is at most $O(N^{7/8})$.

If this conjecture is true, we can prove the following.

Conjecture 2 If **Conjecture 1** is true, this task can be solved in $O(N^{11/8} \log N)$ time on average.

Even in 2022, there are many problems whose computational complexities have been improved. Is the above conjecture true? Is the computational complexity of this task can be improved further? We finish this review by asking these questions.

[*14] It is also called a Fenwick Tree.

[*15] For example, we ask whether "there is a possibility that it is used in a LIS" if $p_{n/2}$ gets closer to $n/2$ to a certain amount. If $p_{n/2} = n/2 + x$, among $p_1, p_2, \ldots, p_{n/2-1}$, there are approximately $n/2 + x/2$ elements which are smaller than $p_{n/2}$. Also, among $p_{n/2+1}, p_{n/2+2}, \ldots, p_n$, there are approximately $n/2 - x/2$ elements which are larger than $p_{n/2}$. Therefore, the expected value of the length of a LIS which contains $p_{n/2}$ is $\sqrt{n/2 + x/2} + \sqrt{n/2 - x/2}$. But there is a stochastic variation of the size, which is approximately equal to $n^{1/4}$. When $x = O(n^{7/8})$, the increment of the expected value of the length is equal to the stochastic variation. From this, we may argue that if the distance is longer than this value, then the element will not be used in a LIS. We can prove the claim in this way.